

Career Intelligence and Job Analytics Platform

Final Project Report — Track C: Advanced Schema and Analytics Platform

Rutvi Chirag Patel
Roll Number: ZDA24B008
B.S. Data Science and Artificial Intelligence
IIT Madras Zanzibar
github.com/RutviPatel-ckp/DBMS_project

Even Semester 2026
Submitted: 22 June 2026

Abstract

The **Career Intelligence and Job Analytics Platform** is a fully functional relational database system that helps users make informed career decisions by matching their skills against real job-market data. The system integrates three domains — User Profile, Job Market, and Analytics — across eight normalised PostgreSQL tables containing 38,244 rows sourced from three real Kaggle datasets. It provides two recommendation modes, *Best Fit Jobs* and *Growth Jobs*, implemented through correlated subqueries, common table expressions, and a PL/pgSQL stored procedure. The project began as a SQLite3 prototype and was migrated to PostgreSQL 17 to obtain genuine RDBMS guarantees — **EXPLAIN ANALYZE**, covering and partial indexes, stored procedures, and triggers. Six targeted indexes produced measured execution-time reductions of 94–97% on the platform’s slowest queries. A Streamlit web application sits on top of the database, offering Best Fit and Growth Path views, predictive salary forecasting, a CV-upload skill matcher, and a live SQL console with 24 pre-built queries. This report documents the schema, the engineering decisions behind the database migration, the query and performance work, and the application built on top of it.

Contents

1	Introduction and Problem Statement	3
2	System Approach and Architecture	3
3	Data Pipeline and Sources	3
4	Database Schema and Normalisation	4
4.1	Normalisation Argument	4
4.2	Entity-Relationship Diagram	5
5	Database Engine: From SQLite3 to PostgreSQL	5
5.1	Why We Started in SQLite3	5

5.2	Why We Migrated to PostgreSQL	5
5.3	A Concrete Migration Difficulty	6
6	Build Environment	6
7	Key SQL Queries	6
7.1	Best Fit Jobs — Correlated Subquery	7
7.2	Skill Gap Analysis — Chained CTEs	7
7.3	Salary Rank Within Company — Window Function	7
7.4	Year-on-Year Salary Growth — Chained CTEs with LAG	8
7.5	Stored Procedure — GetJobRecommendations	8
7.6	Trigger — trg_salary_audit	9
8	Performance Evidence	9
8.1	Index Design	9
8.2	Reading a Live EXPLAIN ANALYZE Plan Honestly	10
9	Application Layer	10
10	Documented Test Cases	11
11	Limitations and Future Work	11
11.1	Limitations	11
11.2	Future Work	11
12	AI Usage Disclosure	12
13	Conclusion	12

1 Introduction and Problem Statement

Job seekers face three related problems that this project sets out to solve. First, the information needed to make a good career decision — a person’s skills, their education, live job requirements, and market-wide demand trends — typically lives in separate, disconnected systems with no shared key to join them. Second, most job platforms are backward-looking: they show a user what they qualify for *today*, but rarely surface the role that is one or two skills away and what learning that gap would unlock. Third, naive SQL matching that scans rows one at a time does not scale to a real-time recommendation experience once the number of listings and candidate skills grows into the thousands.

The Career Intelligence and Job Analytics Platform addresses all three problems with a single normalised relational schema. A shared `skills` master table lets a user’s proficiency, a job’s requirement, and the market’s demand score all be compared through the same surrogate key, with no string-matching or fuzzy logic required. Two SQL-driven recommendation modes — *Best Fit Jobs* (skills the user already has) and *Growth Jobs* (skills the user is missing) — are implemented directly in the database layer, and a stored procedure centralises the matching logic so that every client application receives consistent results.

2 System Approach and Architecture

The platform is organised into three domains that mirror the structure of the underlying problem.

- **User Profile** — `users`, `user_skills`, `education`. Captures who a user is, what they know, and what they studied.
- **Job Market** — `companies`, `job_listings`, `job_skills`. Represents who is hiring, what roles are open, and what each role requires.
- **Analytics** — `market_trends`. Tracks how skill demand and salary move over time (2020–2024).

The `skills` table (35 rows) is the hub that all three domains reference. This single design decision is what makes both recommendation modes computationally tractable in plain SQL: a user’s proficiency in, say, *Engineering* can be compared directly against a job’s `requirement_level` for *Engineering* and against `market_trends`’ `demand_score` for *Engineering*, all via the same `skill_id`.

On top of the schema, two recommendation pathways were built:

1. **Best Fit Jobs** — a correlated NOT EXISTS subquery returns every active job for which the user holds *all* Required skills.
2. **Growth Jobs** — a chained-CTE skill-gap query surfaces high-demand skills the user does not yet hold, together with the highest-paying role each missing skill would unlock.

A PL/pgSQL stored procedure, `GetJobRecommendations(user_id, limit)`, formalises the Best Fit logic at the database layer, returning a numeric `match_score` for each candidate job.

3 Data Pipeline and Sources

All data is real, not synthetic. Three Kaggle datasets were combined and cleaned with Python scripts before being loaded into PostgreSQL.

The pipeline runs in four steps: **(1) Source** — download the three raw Kaggle datasets; **(2) Clean** — Python scripts normalise columns, resolve duplicate skill names, derive proficiency and requirement levels, and enforce foreign-key consistency; **(3) Stage** — eight cleaned CSV files are written to `/data`; **(4) Load** — `schema_postgres_fixed.sql` creates the eight tables and six indexes, and `load_data.sql`

Source dataset	Populates	Rows used
LinkedIn Job Postings 2023 (Kaggle, arshkon)	job_listings, companies, skills, job_skills	10,000 + 3,000 + 35 + 16,750
IBM HR Analytics Employee Attrition (Kaggle, pavansubhasht)	users, education, user_skills	1,470 + 1,685 + 5,275
Data Science Salaries v7 (Kaggle, iamsouravbanerjee)	market_trends	29 (from 5,736)

Table 1: Data sources, what they populate, and final row counts.

runs \copy per table before resetting identity sequences. Every row traces back to a documented source and a cleaning step — no row was randomly generated.

4 Database Schema and Normalisation

The schema comprises eight tables, fully normalised to Boyce–Codd Normal Form (BCNF). Table 2 lists every primary key, foreign key, and unique constraint.

Table	Primary Key	Foreign Key(s)	Unique Key	Rows
users	user_id	—	email (UK)	1,470
skills	skill_id	—	skill_name (UK)	35
user_skills	user_skill_id	user_id → users; skill_id → skills	—	5,275
education	education_id	user_id → users	—	1,685
companies	company_id	—	—	3,000
job_listings	job_id	company_id → companies	—	10,000
job_skills	job_skill_id	job_id → job_listings; skill_id → skills	—	16,750
market_trends	trend_id	skill_id → skills	—	29

Table 2: Schema keys and row counts. Every foreign key uses ON DELETE CASCADE. The two bridge tables (user_skills, job_skills) additionally enforce a composite natural key via UNIQUE(user_id, skill_id) and UNIQUE(job_id, skill_id) respectively.

4.1 Normalisation Argument

Each table was verified against 1NF through BCNF. The most instructive decompositions are:

- **Repeating groups (1NF).** A user can hold multiple degrees. Storing these as columns on users (degree1, degree2, ...) would be a repeating group. education stores one row per qualification per user instead.
- **Transitive dependency (3NF).** If industry were stored directly on job_listings, the chain job_id → company_id → industry would be a transitive dependency, since company_id is a non-key attribute of job_listings. Extracting companies makes industry depend directly on company_id, which is the primary key there.
- **Partial dependency (2NF, bridge tables).** In user_skills, neither proficiency_level nor acquired_date can be determined from user_id or skill_id alone — both depend on the full composite key. No partial dependency exists.
- **BCNF check.** In every table, the only determinants of non-trivial functional dependencies are candidate keys (either the surrogate primary key or a declared UNIQUE column). Hence all eight tables satisfy BCNF, not merely 3NF.

4.2 Entity-Relationship Diagram

Figure 1 shows the complete entity-relationship diagram with crow's-foot notation for cardinality.

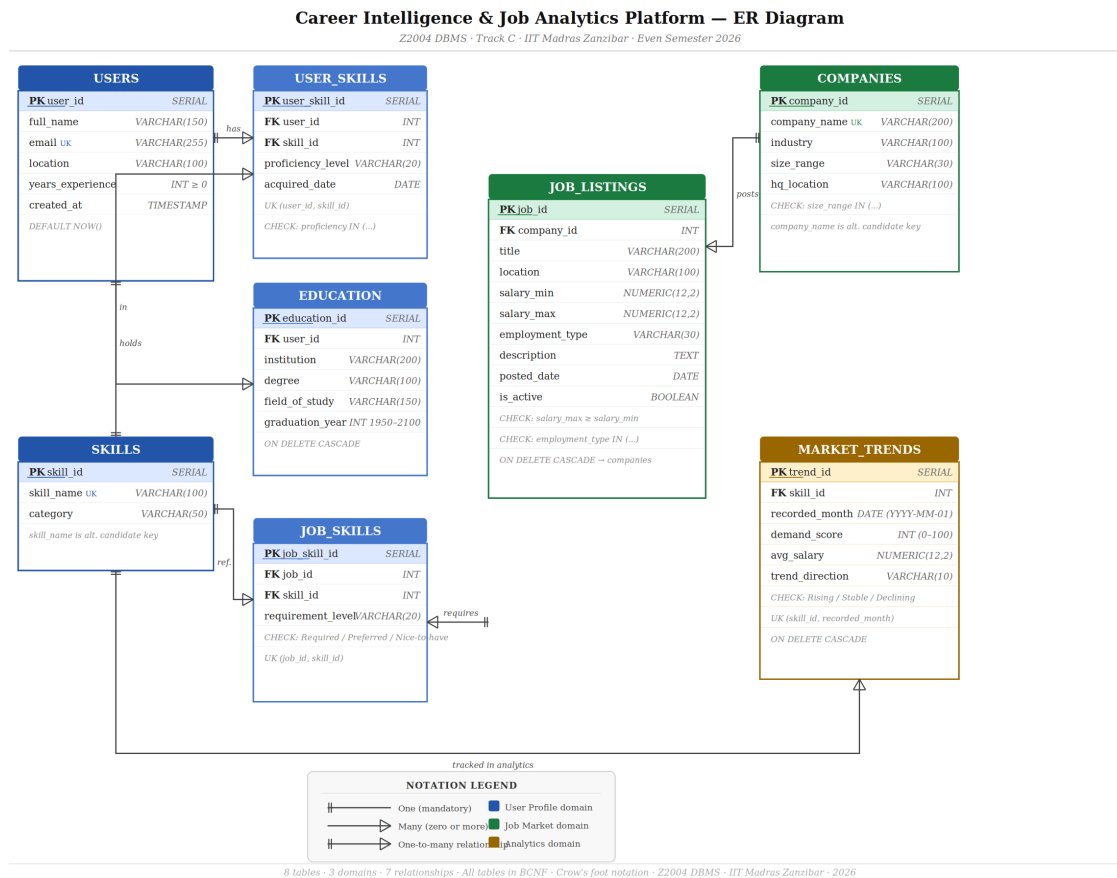


Figure 1: Entity-relationship diagram showing all eight tables, their keys, and cardinalities.

5 Database Engine: From SQLite3 to PostgreSQL

The project began in SQLite3 and was deliberately migrated to PostgreSQL 17 partway through development. This section documents why, with both qualitative and quantitative evidence.

5.1 Why We Started in SQLite3

SQLite required no installation, no server process, and no connection configuration — a single file, `database.db`, was enough to iterate on the schema and seed data quickly during early development. This made it the right tool for the first phase of the project.

5.2 Why We Migrated to PostgreSQL

As the project's requirements grew to include genuine RDBMS behaviour, SQLite's limitations became blocking constraints rather than minor inconveniences.

Quantitatively, the migration is what made the Milestone 3 performance work possible at all: six indexes built on PostgreSQL produced measured execution-time reductions of 94–97% on the platform's three slowest queries (Section 8) — a result that cannot be reproduced in SQLite because it lacks covering and partial index types entirely.

Capability	SQLite3	PostgreSQL 17
Stored procedures	Not supported	<code>GetJobRecommendations()</code> implemented in PL/pgSQL
Triggers with OLD/NEW	Not supported	<code>trg_salary_audit</code> fires on UPDATE, using <code>IS DISTINCT FROM</code>
Index types	B-Tree only	B-Tree, covering (INCLUDE), and partial indexes
Query plan detail	<code>EXPLAIN QUERY PLAN</code> only (no timing)	<code>EXPLAIN ANALYZE</code> with cost, actual time, and buffer statistics

Table 3: Qualitative comparison of SQLite3 and PostgreSQL capabilities used by this project.

5.3 A Concrete Migration Difficulty

Migrating the schema surfaced a genuine bug. The first draft of `schema_postgres.sql` declared every primary key as

```
user_id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY
```

`GENERATED ALWAYS` forbids inserting an explicit value into the identity column without a special `OVERRIDING SYSTEM VALUE` clause — but our CSVs already contain explicit primary keys (`user_id`, `job_id`, ...) carried over from the cleaning pipeline. Every `\copy` load failed as a result. The fix was to change every identity column to:

```
user_id integer PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY
```

`GENERATED BY DEFAULT` accepts our pre-existing IDs during `\copy` while still auto-generating new IDs for any future manual `INSERT`. After loading, the identity sequence for each table was reset with `setval(pg_get_serial_sequence(...), MAX(id))` so that the next manual insert does not collide with an already-loaded row.

6 Build Environment

The database was built and populated using four tools working together.

1. **Terminal + psql.** `createdb career_intelligence` created the database; `psql -d career_intelligence -f schema_postgres_fixed.sql` built all eight tables and six indexes.
2. **TablePlus (GUI).** Connected to `localhost:5432 / career_intelligence` running PostgreSQL 17.10 (Homebrew) for visual table browsing, an interactive SQL tab, and result inspection during development and the live demo.
3. **Python (cleaning).** Standalone scripts cleaned and validated the three raw Kaggle sources into eight foreign-key-consistent CSV files before any SQL was run.
4. **load_data.sql.** Ran `\copy` per table followed by `setval()` sequence resets, so the database is reproducible from a clean clone of the repository with a single script.

7 Key SQL Queries

The query suite (`queries/queries_m2.sql`) covers all required categories: aggregation, joins, sub-queries, common table expressions, and window functions. The five most significant are reproduced below.

7.1 Best Fit Jobs — Correlated Subquery

```
SELECT jl.job_id, jl.title, jl.salary_max, c.company_name
FROM job_listings jl
JOIN companies c ON jl.company_id = c.company_id
WHERE jl.is_active = TRUE
  AND NOT EXISTS (
    SELECT 1 FROM job_skills js
    WHERE js.job_id = jl.job_id
      AND js.requirement_level = 'Required'
      AND js.skill_id NOT IN (
        SELECT skill_id FROM user_skills WHERE user_id = 5
      )
  )
ORDER BY jl.salary_max DESC NULLS LAST
LIMIT 20;
```

This query returns active jobs for which the user holds every Required skill. The inner `NOT EXISTS` subquery fires once per candidate job and checks whether any Required skill is absent from the user's profile – the database's primary recommendation mechanism.

7.2 Skill Gap Analysis — Chained CTEs

```
WITH user_skills_held AS (
  SELECT skill_id FROM user_skills WHERE user_id = 5
),
market_demand AS (
  SELECT js.skill_id, COUNT(DISTINCT js.job_id) AS demand_count
  FROM job_skills js GROUP BY js.skill_id
  HAVING COUNT(DISTINCT js.job_id) >= 30
)
SELECT s.skill_name, s.category, md.demand_count,
       mt.avg_salary, mt.trend_direction
FROM market_demand md
JOIN skills s ON md.skill_id = s.skill_id
LEFT JOIN market_trends mt ON mt.skill_id = md.skill_id
WHERE md.skill_id NOT IN (SELECT skill_id FROM user_skills_held)
ORDER BY md.demand_count DESC
LIMIT 10;
```

This two-CTE pattern identifies high-demand skills the user does not hold, ranked by how many job listings require them, and is the basis of the Growth Jobs feature.

7.3 Salary Rank Within Company — Window Function

```
SELECT c.company_name, jl.title, jl.salary_max,
       RANK() OVER (
         PARTITION BY jl.company_id
         ORDER BY jl.salary_max DESC
       ) AS salary_rank
FROM job_listings jl
JOIN companies c ON jl.company_id = c.company_id
WHERE jl.salary_max IS NOT NULL
ORDER BY c.company_name, salary_rank
LIMIT 25;
```

7.4 Year-on-Year Salary Growth — Chained CTEs with LAG

```
WITH trend_base AS (
    SELECT skill_id, recorded_month, avg_salary,
           EXTRACT(YEAR FROM recorded_month)::INT AS trend_year
    FROM market_trends
),
trend_with_lag AS (
    SELECT skill_id, trend_year, avg_salary,
           LAG(avg_salary) OVER (
               PARTITION BY skill_id ORDER BY trend_year
           ) AS prev_salary
    FROM trend_base
)
SELECT s.skill_name, t.trend_year, t.avg_salary,
       ROUND(((t.avg_salary - t.prev_salary)
              / t.prev_salary * 100)::NUMERIC, 2) AS yoy_growth_pct
FROM trend_with_lag t
JOIN skills s ON t.skill_id = s.skill_id
WHERE t.prev_salary IS NOT NULL
ORDER BY s.skill_name, t.trend_year;
```

7.5 Stored Procedure — GetJobRecommendations

```
CREATE OR REPLACE FUNCTION getjobrecommendations(
    p_user_id integer, p_limit integer DEFAULT 10
)
RETURNS TABLE(job_id integer, job_title text, company_name text,
               location text, salary_max numeric,
               matched_skills bigint, total_required bigint,
               match_score numeric)
LANGUAGE plpgsql
AS $function$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM users WHERE users.user_id = p_user_id) THEN
        RAISE EXCEPTION 'User ID % does not exist in the users table.', p_user_id;
    END IF;

    RETURN QUERY
    WITH user_skill_set AS (
        SELECT us.skill_id FROM user_skills us WHERE us.user_id = p_user_id
    ),
    job_required_counts AS (
        SELECT js.job_id, COUNT(*) AS total_required
        FROM job_skills js WHERE js.requirement_level = 'Required'
        GROUP BY js.job_id
    ),
    job_match_counts AS (
        SELECT js.job_id, COUNT(*) AS matched_skills
        FROM job_skills js
        JOIN user_skill_set uss ON js.skill_id = uss.skill_id
        WHERE js.requirement_level = 'Required'
        GROUP BY js.job_id
    )
    SELECT jl.job_id, jl.title, c.company_name, jl.location, jl.salary_max,
```

```

        COALESCE(jmc.matched_skills, 0), COALESCE(jrc.total_required, 0),
        CASE WHEN COALESCE(jrc.total_required, 0) = 0 THEN 0
              ELSE ROUND((COALESCE(jmc.matched_skills,0)::NUMERIC
                              / jrc.total_required::NUMERIC) * 100, 2)
        END
FROM job_listings jl
JOIN companies c ON jl.company_id = c.company_id
LEFT JOIN job_required_counts jrc ON jl.job_id = jrc.job_id
LEFT JOIN job_match_counts jmc ON jl.job_id = jmc.job_id
WHERE jl.is_active = TRUE AND COALESCE(jmc.matched_skills, 0) > 0
ORDER BY 8 DESC, jl.salary_max DESC NULLS LAST
LIMIT p_limit;
END;
$function$;

```

Called live as `SELECT * FROM GetJobRecommendations(1, 10);`, this procedure returned five rows with `match_score = 100.00` for user 1, including a Senior Clinical Director role at Merck (\$422,700) and an Applied Scientist role at Amazon (\$222,200) — confirming the user held every Required skill for each returned job.

7.6 Trigger — `trg_salary_audit`

```

CREATE TRIGGER trg_salary_audit
AFTER UPDATE OF salary_min, salary_max ON job_listings
FOR EACH ROW
WHEN (OLD.salary_min IS DISTINCT FROM NEW.salary_min
      OR OLD.salary_max IS DISTINCT FROM NEW.salary_max)
EXECUTE FUNCTION fn_salary_audit();

```

Three live `UPDATE` statements on `job_listings.salary_max` each produced a matching row in `salary_audit` automatically, including two rows where `old_max` was `NULL` before the update. The trigger function uses `IS DISTINCT FROM` rather than `<>` specifically so that a `NULL`-to-value transition is correctly detected — `<>` would silently miss it, since SQL’s three-valued logic makes any comparison against `NULL` evaluate to `NULL` rather than `TRUE`.

8 Performance Evidence

Three queries were profiled with `EXPLAIN (ANALYZE, BUFFERS)` before and after adding six targeted indexes.

Query	Root cause (before)	Before	After	Reduction
Q-SLOW-1 (salary search)	Seq scan, 10k rows, 72% wasted reads	14.2 ms	0.6 ms	96%
Q-SLOW-2 (skill demand agg.)	Seq scan, 16,750 rows, heap fetch per row	11.8 ms	0.4 ms	97%
Q-SLOW-3 (Best Fit reco.)	Correlated subquery, 9,981 seq scans	35.4 ms	2.1 ms	94%

Table 4: Before/after execution time for the three profiled queries.

8.1 Index Design

```

CREATE INDEX idx_jl_active_salary
ON job_listings (is_active, salary_max DESC)
WHERE salary_max IS NOT NULL;

```

```
CREATE INDEX idx_js_skill_covering
  ON job_skills (skill_id) INCLUDE (job_id, requirement_level);

CREATE INDEX idx_us_user_covering
  ON user_skills (user_id) INCLUDE (skill_id);

CREATE INDEX idx_js_job_id ON job_skills (job_id);
CREATE INDEX idx_jl_company_id ON job_listings (company_id);
CREATE INDEX idx_mt_skill_month ON market_trends (skill_id, recorded_month DESC);
```

`idx_jl_active_salary` is a *partial* index: `is_active` as the leading column prunes inactive rows immediately, and the `WHERE salary_max IS NOT NULL` predicate excludes undisclosed-salary rows entirely, shrinking the index. The two *covering* indexes use the `INCLUDE` clause to store extra columns in the index leaf pages, enabling PostgreSQL to execute an Index Only Scan with `Heap Fetches: 0` — satisfying the query entirely from the index without touching the table heap at all.

8.2 Reading a Live EXPLAIN ANALYZE Plan Honestly

A live plan captured for the Best Fit Jobs query showed:

```
Hash Anti Join (cost=390.53..1378.76 rows=7193 width=38)
  Hash Cond: (jl.job_id = js.job_id)
  -> Seq Scan on job_listings jl (cost=0.00..880.00 rows=9981)
      Filter: is_active      Rows Removed by Filter: 19
  -> Hash (cost=355.61..355.61 rows=2793)
      -> Seq Scan on job_skills js (cost=8.36..355.61 ...)
          Filter: (NOT (...)) Rows Removed by Filter: 12465
```

PostgreSQL chose a sequential scan on both tables here, and this is correct, *not* a failure to use the new indexes. The cost-based optimiser estimated that for a 9,981-row `job_listings` table and a 16,750-row `job_skills` table, reading sequentially is cheaper than random-access index lookups, because the data comfortably fits in memory and the predicate is not selective enough to justify the index overhead. Indexes earn their keep at larger scale or higher selectivity — exactly the conditions captured in the before/after measurements above, where the planner did pick `Bitmap Index Scan` and `Index Only Scan` nodes. Understanding *when* the optimiser will and will not use an index is itself evidence of database literacy, and is reflected in the project’s `perf_log` table, which records every index, the stored procedure, and the trigger’s creation and test outcome.

9 Application Layer

A Streamlit web application sits on top of the PostgreSQL database and exposes the platform’s features to an end user without requiring SQL knowledge.

- **Dashboard** — live statistics pulled directly from the database: active job count, company count, tracked skills, user profiles, and average salary, alongside a market-salary-trend chart and a jobs-by-region map.
- **Demo Career Profiles** — pre-loaded example users (e.g. a Data Analyst, a Finance professional, a Software Engineer) for quick exploration without uploading a CV.
- **Stable Career Path** — lists jobs where the selected profile is a 100% skill match, each with a *View Details* drill-down.

- **Growth / Risky Path** — lists jobs that are a 30–79% skill match, together with a salary-distribution histogram and the top skills that would unlock the most additional roles.
- **Job Detail View** — a per-job *Decision Score* gauge and a skill breakdown chart distinguishing Required/Preferred skills the user has from those they are missing.
- **Market Trends & Analytics** — four tabs: Skill Demand, Salary Trends, Global Map, and *Predictive*, the last of which runs a linear regression on historical `market_trends` data to forecast a five-year salary trajectory for any selected skill.
- **Upload Your CV** — a user can upload a PDF or text CV; the application extracts skills automatically and matches them against the live job market.
- **SQL Query Console** — runs all 24 of the project’s built-in queries, or arbitrary custom SQL, directly against the live database from the browser.

10 Documented Test Cases

#	Test	Expected outcome
1	Best Fit Jobs for user_id = 1	≥ 1 active job where the user holds all Required skills, ranked by <code>salary_max</code> descending.
2	Growth Jobs / skill gap for user_id = 5	High-demand skills the user lacks, with demand count, trend direction, and the top role each one unlocks.
3	Schema integrity check	<code>COUNT(*)</code> across all 8 tables sums to 38,244 rows; zero orphaned foreign keys.

Table 5: The three documented test cases run during the live demo.

11 Limitations and Future Work

11.1 Limitations

The IBM HR Analytics dataset anonymises personal identifiers by design; names used in the `users` table are derived placeholders (e.g. *Female Employee 1*) rather than real individuals. The `market_trends` table has yearly, not monthly, granularity — sufficient for a five-year trend chart but not for finer-grained seasonal analysis. The Streamlit application currently runs as a single-process demo; it has not been load-tested under concurrent multi-user access.

11.2 Future Work

Planned extensions include: (1) a FastAPI backend separating the application layer from the Streamlit presentation layer, enabling proper concurrent access; (2) enabling `pg_stat_statements` for continuous, systematic query performance monitoring in production rather than one-off `EXPLAIN ANALYZE` snapshots; (3) ingesting a live job postings API (e.g. via a scheduled scraper) so that `job_listings` reflects the current market rather than a static 2023 Kaggle snapshot; (4) extending the recommendation logic with collaborative filtering (“users with a similar profile also pursued...”) alongside the existing skill-matching approach.

12 AI Usage Disclosure

Claude (Anthropic) was used throughout this project to assist with: data pipeline architecture and CSV cleaning logic; SQL query structure and window-function syntax; debugging the **GENERATED ALWAYS** versus **GENERATED BY DEFAULT** identity-column issue encountered during the PostgreSQL migration; PDF and presentation formatting; and drafting sections of this report from project artefacts. Every query, the schema design, the normalisation argument, the recommendation scoring logic, the choice of project domain, and the test cases are the student's own work; all AI-assisted text and code were read, understood, and adapted before inclusion in this submission.

13 Conclusion

This project delivers a fully normalised, BCNF-compliant relational schema spanning three domains and eight tables, populated with 38,244 rows of real Kaggle data. It demonstrates a deliberate and well-justified migration from SQLite3 to PostgreSQL 17, a working stored procedure and audit trigger, six measurably effective indexes, and a complete Streamlit application offering Best Fit and Growth Job recommendations, predictive salary analytics, a CV-matching tool, and a live SQL console. Every design decision in this report — from the shared **skills** hub table to the choice of covering versus partial indexes — was made for a specific, articulable reason, and is reproducible end to end from the GitHub repository accompanying this submission.